VerveFlo

Pneumatic Compression Therapy Device

Complete Technical & User Documentation | v3.0

Platform	ESP32-S3 · Arduino IDE
Firmware	verveflo.ino — Single file, all-in-one
Display	SH1106 OLED 128x64 I2C
Sensor	HX710B Pressure Sensor
WiFi	AP Captive Portal + STA Mode
Company	Solvin Medical Devices

This document covers hardware setup, pin mapping, firmware architecture, WiFi provisioning, web UI, pressure sensor calibration, serial debug commands, and complete source code for the VerveFlo compression therapy device.

Table of Contents

1. Device Overview

VerveFlo is a pneumatic sequential compression therapy device. It inflates five cuffs in a programmed sequence to push venous blood and lymphatic fluid from the distal extremity (foot/ankle) toward the proximal (thigh/groin), mimicking the natural muscle pump mechanism.

Clinical Applications

Application	Description
DVT Prevention	Prevents deep vein thrombosis in bedridden or post-surgery patients
Lymphedema	Reduces fluid buildup in limbs via sequential lymphatic drainage
Sports Recovery	Flushes lactic acid, improves circulation after exercise
Post-op Rehab	Used after orthopedic surgeries — knee, hip replacements

How It Works

5 cuffs wrap around the leg. They inflate sequentially distal to proximal, each cuff held at target pressure (40/70/100 mmHg) for a set time (default 8s), then deflated via a release valve before the next cuff fires. The pump builds pressure continuously while the session runs.

Therapy Modes

Mode	Name	Sequence	Use Case
1	Distal to Proximal	C1→C2→C3→C4→C5	Standard venous return
2	Proximal to Distal	C5→C4→C3→C2→C1	Reverse lymphatic drainage
3	Alternating	C1→C3→C5→C2→C4	Deep vessel targeting
4	Wave	C1→C5→C2→C4→C3	Enhanced circulation
Custom	User Defined	Any order	Via web UI mode builder

2. Hardware & Pin Mapping

Microcontroller: ESP32-S3 Dev Module

All GPIO assignments verified from hardware schematic.

Valve & Actuator Pins

Component	Type	GPIO	Notes
Fill Valve 1 (Cuff 1)	Solenoid PWM	GPIO 26	Foot/distal
Fill Valve 2 (Cuff 2)	Solenoid PWM	GPIO 47	
Fill Valve 3 (Cuff 3)	Solenoid PWM	GPIO 39	Mid leg
Fill Valve 4 (Cuff 4)	Solenoid PWM	GPIO 40	
Fill Valve 5 (Cuff 5)	Solenoid PWM	GPIO 41	Thigh/proximal
Release Valve	Solenoid PWM	GPIO 42	Deflates cuffs
Pump Motor	DC Motor PWM	GPIO 21	5V compatible, soft start

Sensor & Display Pins

Component	GPIO	Signal	Notes
HX710B Data (Dout)	GPIO 34	Input	Pressure sensor data
HX710B Clock (SCK)	GPIO 33	Output	Pressure sensor clock
OLED SH1106	I2C HW	I2C	SDA/SCL hardware I2C
Battery ADC	GPIO 15	ADC	100k+100k divider, ratio 2.0
Status LED	GPIO 3	Output	Debug indicator

Button Pins

All buttons active LOW with internal pull-up.

Button	GPIO	Short Press	Long Press (2s)
BTN Start / Play	GPIO 35	Start / Pause / Resume	Emergency Release + Reset
BTN Mode	GPIO 36	Cycle Mode 1-4	Only in READY/PAUSED state
BTN Pressure	GPIO 37	Cycle LOW/MED/HIGH	Only in READY/PAUSED state
BTN Time	GPIO 48	Cycle 10/20/30 min	Only in READY/PAUSED state

3. Power Architecture

VerveFlo uses a 2S 18650 Li-Ion battery pack (8.4V fully charged). Two separate buck converters regulate to 5V and 3.3V rails.

Rail	IC	Powers	Notes
8.4V (raw)	2S 18650	Buck inputs	Fully charged 8.4V, cutoff ~6V
5V	TPS54302DDCR	Valves + Pump Motor	3A rated, min input 4.5V
3.3V	Separate buck	ESP32-S3, OLED, Buttons	Logic rail

TPS54302DDCR Specifications

Parameter	Value
Input Voltage Range	4.5V to 28V
Output Current	3A continuous
Package	SOT-23-6
Switching Frequency	570 kHz
Safe for 2S 18650?	YES — 8.4V well within range

WARNING: Motor startup current spike can damage the buck. Firmware implements PWM soft start (0 to 255 over ~1.5s, 5 counts per step every 30ms). Add 100uF capacitor across 5V output for additional spike absorption.

Battery BMS Note

BMS may shut off prematurely due to voltage sag under motor load. This is caused by the motor startup current spike pulling the battery voltage below the BMS cutoff threshold momentarily. The PWM soft start ramp in firmware significantly reduces this. A 1000uF+ cap directly on battery output terminals eliminates it completely.

4. Libraries & Arduino IDE Setup

Required Libraries

Library	Author	Install Via	Version Tested
U8g2	olikraus	Manage Libraries → search U8g2	2.36.2
Queuetue_HX711_Library	bogde	Manage Libraries → search Q2HX711	1.0.2
WiFi	Espressif	Built into ESP32 core — no install	3.3.8
WebServer	Espressif	Built into ESP32 core — no install	3.3.8
DNSServer	Espressif	Built into ESP32 core — no install	3.3.8
Preferences	Espressif	Built into ESP32 core — no install	3.3.8

Board Manager Setup

Add this URL in File → Preferences → Additional Boards Manager URLs:

```
https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json
```

Then: Tools → Board → Boards Manager → search esp32 → install esp32 by Espressif Systems

Board Settings

Setting	Value
Board	ESP32S3 Dev Module
USB CDC On Boot	Enabled (CRITICAL for Serial Monitor)
USB Mode	Hardware CDC and JTAG
Partition Scheme	Default 4MB with spiffs
Upload Speed	921600
CPU Frequency	240MHz
Flash Size	4MB

USB CDC On Boot MUST be Enabled. Without it Serial Monitor shows nothing.

5. Firmware Architecture

State Machine

The entire firmware runs on a non-blocking state machine. No delay() calls in the main loop. All timing uses millis().

State	Description	LED Pattern
STATE_BOOT	Boot animation — SOLVIN logo + loading bar	Fast blink 5Hz
STATE_READY	Idle, waiting for Start button or web command	Slow blink 1Hz
STATE_RUNNING	Active session — cuffs inflating in sequence	Double pulse
STATE_PAUSED	Session paused, pressure held	Slow blink 1Hz
STATE_DEFLATE_WAIT	2s gap between cuffs during deflation	Double pulse
STATE_RELEASING	Sequential release of all cuffs	Fast blink 5Hz

Session Flow

```
STATE_READY
    → BTN_START or web /start
STATE_RUNNING
    → Open valve for cuff N (PWM ramp 0→255 over 2.5s)
    → Wait for pressure >= target (40/70/100 mmHg)
    → Hold for g_holdMs (default 8000ms)
    → Close inflate valve + open release valve
STATE_DEFLATE_WAIT
    → Wait g_deflateWaitMs (default 2000ms)
    → Close release valve
    → Advance to next cuff → back to STATE_RUNNING
    → After all 5 cuffs → check if session time expired
    → If expired → STATE_RELEASING
STATE_RELEASING
    → Release each cuff sequentially (3s each)
    → Close release valve → STATE_READY
```

Valve PWM Ramp

Solenoid valves are driven via analogWrite PWM. Opening ramps 0 to 255 over 2500ms (256 steps, ~9ms per step). Closing ramps 255 to 0. This prevents voltage spikes and gives a smooth kick+hold behavior. Only one valve ramps at a time via a single ValveRamp struct.

Motor Soft Start

Pump motor uses analogWrite with a software ramp: 5 PWM counts per step every 30ms = approximately 1.5 seconds from 0 to full speed. This prevents inrush current spikes that kill the buck converter.

Pressure Reading

```
// Formula:
mmHg = CALIB_M * (raw - CALIB_ZERO) + CALIB_B

// Current calibrated values:
const float CALIB_M  =  0.00000625f;  // positive — raw increases with pressure
const float CALIB_B  =  0.0000f;
long        CALIB_ZERO = 8388000L;    // raw value at atmosphere (0 mmHg)

// Overflow filter: HX710B saturates to ~16777216 during fast release
if (raw > 16000000L || raw < 0) return last_good_value;
```

Key Timing Parameters

Parameter	Default	Serial Command	Range
Hold per cuff	8000ms	hold N (seconds)	2-60s
Deflate gap	2000ms	gap N (seconds)	1-15s
Release per cuff	3000ms	Not configurable	Fixed
Valve ramp time	2500ms	Not configurable	Fixed
Motor ramp time	~1500ms	Not configurable	Fixed
Pressure sample	200ms	Not configurable	Fixed
Display update	250ms	Not configurable	Fixed
Battery read	10s	Not configurable	Fixed

6. OLED Display Screens

Display: SH1106 128x64 I2C. All screens include 4-bar battery icon top-right.

Screen	Content
Boot — SOLVIN Logo	Letters S-O-L-V-I-N appear one by one. "Medical Devices" fades in. ~1 second.
Boot — VerveFlo Loading	"VerveFlo / Compression Therapy". Loading bar fills with status messages: Initialising / Calibrating / Loading mode
READY	MODE N + mode name. PRES LOW/MED/HIGH. TIME 10m/20m/30m. WiFi IP address. Blinking [PRESS START]
RUNNING	Large pressure number (mmHg) + target. Countdown timer mm:ss. Mode name. 5 animated circles — active cuff p
DEFLATING	Same as RUNNING but header shows DEFLATING instead of RUNNING.
PAUSED	Blinking large PAUSED text. Press START to resume. Mode/pressure/time settings reminder.
RELEASING	Progress bar filling as each cuff releases. Cuff X/5 counter. Animated circles. Please wait...

7. WiFi Provisioning

VerveFlo manages WiFi credentials automatically with zero configuration needed by the user.

First Boot — AP Mode

```
1. No saved WiFi credentials in flash
2. ESP32 starts hotspot: "VerveFlo-Setup" (open, no password)
3. DNS server redirects ALL traffic to 192.168.4.1 (captive portal)
4. User connects phone to "VerveFlo-Setup"
5. Browser auto-opens (captive portal) OR user opens 192.168.4.1
6. Setup page shows WiFi scan list
7. User taps network → enters password → taps CONNECT & SAVE
8. Credentials saved to ESP32 flash (Preferences library)
9. Device restarts → connects to home WiFi
10. IP address shown on OLED and Serial Monitor
```

Subsequent Boots — STA Mode

```
1. Reads saved SSID + password from flash
2. Connects to home WiFi (24 attempts × 500ms = 12 second timeout)
3. If connected → shows IP on OLED → WebServer starts on port 80
4. If failed → falls back to AP mode automatically
```

Reset WiFi

```
Serial command:  wificlear
Web endpoint:    http://<IP>/wificlear
Effect: Clears saved credentials, restarts device in AP mode
```

Access Points

Endpoint	Method	Description
/	GET	Serve HTML web UI
/status	GET	JSON status — polled every 500ms by UI
/start	GET	Start session
/pause	GET	Pause or resume session
/stop	GET	Stop + release all cuffs
/release	GET	Emergency release
/set_pressure	GET	?level=0/1/2 (LOW/MED/HIGH)
/set_time	GET	?idx=0/1/2 (10/20/30 min)
/set_mode	GET	?mode=0-3

/set_timings	GET	?hold_ms=N&gap_ms=N
/scan	GET	Scan WiFi networks (AP mode setup)
/savewifi	GET	?ssid=X&pass=Y — save credentials
/wificlear	GET	Clear WiFi, restart in AP mode

8. Web UI

The web UI is embedded directly in firmware as a PROGMEM string — no SPIFFS upload needed. Served at http://[device-IP]/ Works in any phone or laptop browser on the same WiFi network.

Features

Feature	Description
Live Status	State chip, animated cuff circles, pressure bar, countdown timer
4-Bar Battery	Visual battery indicator synced with device
START/PAUSE/RESUME	Single morphing button changes color and label with state
STOP + RELEASE	Separate stop and emergency release buttons
Pressure Target	LOW/MED/HIGH chips (40/70/100 mmHg)
Session Time	10/20/30 min chips
Hold/Gap Sliders	Adjustable inflate hold (3-30s) and deflate gap (1-15s)
Mode Builder	M1-M4 presets + custom sequence builder (tap cuffs in order)
WiFi Setup Page	Shown in AP mode — scan networks, enter password, save
Demo Mode	Works without device connected — simulates session for UI testing
Auto Connect	Automatically connects to device IP from URL hostname

How to Access

```
1. Connect phone/laptop to same WiFi as VerveFlo
2. Open Serial Monitor → type "ip" → note the IP address
   (also shown on OLED ready screen)
3. Open browser → type: http://192.168.x.x
4. UI loads automatically
5. Type IP in CONNECT field → tap CONNECT → status goes LIVE
```

9. Pressure Sensor Calibration

Sensor: HX710B connected to GPIO 34 (Dout) and GPIO 33 (SCK).

Key Findings from Hardware Testing

Finding	Detail
Raw direction	Raw value INCREASES with pressure (not decreases as initially assumed)
Atmosphere raw	Approximately 8,388,000 at 0 mmHg
Pressure raw	Increases toward ~8,900,000 at higher pressures
Overflow	~16,777,216 (2^24) during fast pressure release — must be filtered
CALIB_M sign	Positive (+0.00000625) because raw goes up with pressure

Calibration Formula

```
mmHg = CALIB_M * (raw - CALIB_ZERO) + CALIB_B

Where:
  raw        = HX710B raw reading
  CALIB_ZERO = raw value at 0 mmHg (atmosphere)
  CALIB_M    = slope (positive for this sensor)
  CALIB_B    = offset (0 after proper zeroing)
```

Current Calibration Values

```
const float CALIB_M   =  0.00000625f;   // slope — positive
const float CALIB_B   =  0.0000f;        // offset
long        CALIB_ZERO = 8388000L;       // atmosphere baseline
                                         // UPDATE this after running 'zero' command
```

Calibration Procedure

```
STEP 1: Upload verveflo_pressure_cal.ino (see Section 13)
STEP 2: Open Serial Monitor at 115200 baud, Newline line ending

STEP 3: At atmosphere (open air, no pressure applied):
   Type:  z
   Result: "Zero set to raw=8XXXXXXX"
   Note this number → put in CALIB_ZERO

STEP 4: Build pressure with reference gauge connected:
   Type:  pump on
   Type:  ps    (watch live pressure stream)
   Type:  pump off   (stop at stable known pressure)
```

```
STEP 5: With reference gauge at e.g. 50 mmHg:
   Type:  c 50
   Result: prints CALIB_M and CALIB_B values

STEP 6: Verify:
   Type:  ps    (should read ~50 mmHg)
   Type:  pump on / pump off to verify range

STEP 7: Copy values to verveflo.ino:
   const float CALIB_M = [value from step 5];
   const float CALIB_B = [value from step 5];
   long CALIB_ZERO = [value from step 3];
```

Live Zero (without reflashing)

```
In main firmware serial monitor:
  Type: zero
  Effect: Samples 16 readings at current pressure, sets CALIB_ZERO live
  Use: Run at atmosphere before each session if readings drift
```

10. Serial Debug Commands

Connect USB cable. Open Arduino IDE Serial Monitor. Set baud rate to 115200. Set line ending to Newline. Type command and press Enter.

Session Control

Command	Action
start	Start therapy session (only in READY state)
pause	Pause session (type again to resume)
stop	Stop session + sequential cuff release
release	Emergency release all cuffs immediately

Status & Readings

Command	Output
status	Full system dump — state, pressure, battery, mode, timing, WiFi IP
pressure	Current pressure in mmHg
battery	Battery voltage, percent, bar count
valves	All 6 valve states — IDLE / RAMP UP / RAMP DN
buttons	Current button pin states HIGH/LOW
pins	All output pin states
ip	Current WiFi IP address or AP mode info
zero	Set pressure zero at current atmosphere reading

Direct Hardware Control (Debug)

Command	Action
v1 on	Cuff 1 valve GPIO26 instant ON (255 PWM)
v1 off	Cuff 1 valve GPIO26 OFF
v1 open	Cuff 1 valve PWM ramp open (2.5s ramp)
v1 close	Cuff 1 valve PWM ramp close
v1 128	Set cuff 1 to specific PWM duty (0-255)
v2-v5 on/off	Same as v1 for cuffs 2-5
vr on/off	Release valve GPIO42 ON/OFF
pump on	Pump motor ON (via soft start ramp)
pump off	Pump motor OFF (via soft stop ramp)
motor 128	Set motor to specific PWM duty 0-255 (with ramp)
alloff	Immediately kill pump + all 6 valves

Diagnostic Tests

Command	Action
selftest	Full auto test: LED blink, battery, pressure sensor, pump 2s, all valves 1s each
pstream	Live pressure stream with bar graph — any key to stop
ptest	Pump ON + watch pressure build to target — auto stops at target
vsweep	Open each inflate valve 2s in sequence — any key to stop

Settings (READY or PAUSED state only)

Command	Action
mode 1-4	Set therapy mode 1=Distal 2=Proximal 3=Alternating 4=Wave
pset 0	Pressure target LOW 40 mmHg
pset 1	Pressure target MED 70 mmHg
pset 2	Pressure target HIGH 100 mmHg
time 0	Session duration 10 minutes
time 1	Session duration 20 minutes
time 2	Session duration 30 minutes
hold N	Cuff inflate hold time in seconds (2-60)
gap N	Deflate gap between cuffs in seconds (1-15)
led on	Force LED solid ON
led off	Force LED OFF
led blink	Blink LED 5x to confirm GPIO3 working
led auto	Restore automatic LED pattern
wificlear	Clear saved WiFi credentials, restart in AP mode
reboot	Software restart
help	Print full command list

11. Complete Main Firmware — verveflo.ino

Single .ino file. Place in folder named verveflo/. Update WIFI_SSID and WIFI_PASS with your network credentials before flashing, or leave blank and use the captive portal WiFi setup on first boot.

NOTE: Full code is too large to reproduce here in its entirety. Key sections are shown below. The complete file was provided during development and is saved as verveflo.ino.

File Header & Includes

```
#include <Arduino.h>
#include <WiFi.h>
#include <WebServer.h>
#include <DNSServer.h>
#include <Preferences.h>
#include <U8g2lib.h>
#include <Wire.h>
#include <Q2HX711.h>

// *** CHANGE THESE TO YOUR WIFI CREDENTIALS ***
const char* WIFI_SSID = "YourWiFiName";
const char* WIFI_PASS = "YourWiFiPassword";
```

Pin Definitions

```
const uint8_t VALVE_PINS[6] = { 26, 47, 39, 40, 41, 42 };
#define RELEASE_VALVE_IDX  5
#define NUM_INFLATE_VALVES 5
#define PUMP_PIN    21
#define LED_PIN     3
#define HX_DATA     34
#define HX_CLK      33
#define BATT_PIN    15
#define BTN_START   35
#define BTN_MODE    36
#define BTN_PRESS   37
#define BTN_TIME    48
```

Calibration Values

```
const float CALIB_M   =  0.00000625f;   // UPDATE after calibration
const float CALIB_B   =  0.0000f;
long        CALIB_ZERO = 8388000L;        // UPDATE after running zero command
float g_pressure = 0.0f;
```

System State Enum

```
enum SystemState {
  STATE_BOOT,
  STATE_READY,
  STATE_RUNNING,
  STATE_PAUSED,
  STATE_RELEASING,
  STATE_DEFLATE_WAIT
};
SystemState g_sysState = STATE_BOOT;
```

Session Settings

```
const float    PRESSURE_LEVELS[] = { 40.0f, 70.0f, 100.0f };
const char*    PRESSURE_LABELS[] = { "LOW", "MED", "HIGH" };
const uint16_t TIME_OPTIONS[]    = { 10, 20, 30 };
const char*    TIME_LABELS[]     = { "10m", "20m", "30m" };

const uint8_t MODE_SEQS[4][5] = {
  { 0,1,2,3,4 },   // Mode 1: Distal to Proximal
  { 4,3,2,1,0 },   // Mode 2: Proximal to Distal
  { 0,2,4,1,3 },   // Mode 3: Alternating
  { 0,4,1,3,2 }    // Mode 4: Wave
};

uint8_t  g_pressureIdx   = 1;     // default MED
uint8_t  g_timeIdx       = 0;     // default 10 min
uint8_t  g_mode          = 0;     // default Mode 1
uint32_t g_holdMs        = 8000UL;
uint32_t g_deflateWaitMs = 2000UL;
```

Pressure Read Function

```
float readPressure() {
  long raw = hx711.read();
  // Filter HX710B overflow during fast pressure release
  if (raw > 16000000L || raw < 0) {
    return g_pressure;  // return last good value
  }
  long  adj    = raw - CALIB_ZERO;
  float mmhg   = CALIB_M * (float)adj + CALIB_B;
  return constrain(mmhg, 0.0f, 200.0f);
}
```

Motor Soft Start

```
// Global motor state
uint8_t       g_motorDuty    = 0;
uint8_t       g_motorTarget  = 0;
bool          g_motorRamping = false;
unsigned long g_motorRampT   = 0;

void setPump(bool on) {
  g_motorTarget  = on ? 255 : 0;
  g_motorRamping = true;
  g_motorRampT   = millis();
  Serial.printf("[PUMP] %s -> ramping\n", on ? "ON" : "OFF");
}

void updateMotorRamp() {
  if (!g_motorRamping) return;
  unsigned long now = millis();
  if (now - g_motorRampT < 30) return;  // 30ms per step
  g_motorRampT = now;
  if (g_motorDuty < g_motorTarget)
    g_motorDuty = min((int)g_motorDuty + 5, (int)g_motorTarget);
  else if (g_motorDuty > g_motorTarget)
    g_motorDuty = max((int)g_motorDuty - 5, (int)g_motorTarget);
  else { g_motorRamping = false; }
  analogWrite(PUMP_PIN, g_motorDuty);  // analogWrite only, no ledc
}
```

Valve PWM Ramp

```
#define RAMP_DURATION_MS   2500UL
#define RAMP_STEPS         256
#define RAMP_STEP_INTERVAL (RAMP_DURATION_MS / RAMP_STEPS)  // ~9ms per step

void startOpenValve(uint8_t idx) {
  g_ramp = {true, true, 0, idx, millis()};
  analogWrite(VALVE_PINS[idx], 0);
}

void startCloseValve(uint8_t idx) {
  // Start at 255 ramp DOWN — prevents uint8_t underflow bug
  g_ramp = {true, false, 255, idx, millis()};
  analogWrite(VALVE_PINS[idx], 255);
}

void updateRamp() {
```

```
  if (!g_ramp.active) return;
  unsigned long now = millis();
  if (now - g_ramp.lastStepTime < RAMP_STEP_INTERVAL) return;
  g_ramp.lastStepTime = now;
  if (g_ramp.opening) {
    if (g_ramp.duty < 255) analogWrite(VALVE_PINS[g_ramp.valveIdx], ++g_ramp.duty);
    else g_ramp.active = false;
  } else {
    if (g_ramp.duty > 0) analogWrite(VALVE_PINS[g_ramp.valveIdx], --g_ramp.duty);
    else { analogWrite(VALVE_PINS[g_ramp.valveIdx], 0); g_ramp.active = false; }
  }
}
```

Main Loop Structure

```
void loop() {
  unsigned long now = millis();

  server.handleClient();          // WiFi web requests
  if (g_apMode) dnsServer.processNextRequest();  // AP captive portal
  handleSerial();                 // Serial commands
  if (now - g_lastBattTime >= BATT_INTERVAL_MS) { readBattery(); }

  if (g_sysState == STATE_BOOT) {
    // Non-blocking boot animation
    if (now - g_bootStepTime >= BOOT_STEP_MS) { drawBootScreen(); }
    updateLED(); return;
  }

  if (now - g_lastPressRead >= PRESS_MS) { g_pressure = readPressure(); }

  pollButtons();
  handleEvents();
  updateRamp();
  updateMotorRamp();              // Motor soft start tick

  if (g_sysState == STATE_RUNNING) updateSession();
  if (g_sysState == STATE_RELEASING || g_sysState == STATE_DEFLATE_WAIT)
    updateReleaseSequence();

  if (now - g_lastDisplay >= DISPLAY_MS) { updateDisplay(); }
  updateLED();
}
```

12. Hardware Test Code — verveflo_hwtest.ino

Standalone test sketch. NO libraries needed (no U8g2, no Q2HX711). Uses raw bit-bang HX710B read. Upload separately to verify hardware before flashing main firmware.

Commands

Command	Action
selftest	Full auto test: LED, battery, pressure sensor, pump 2s, all valves 1s each
led	Blink LED 5x on GPIO 3
pump on	Pump motor ON GPIO 21
pump off	Pump motor OFF
v1 on/off	Fill valve 1 GPIO 26
v2 on/off	Fill valve 2 GPIO 47
v3 on/off	Fill valve 3 GPIO 39
v4 on/off	Fill valve 4 GPIO 40
v5 on/off	Fill valve 5 GPIO 41
vr on/off	Release valve GPIO 42
alloff	Everything OFF immediately
pressure	Single raw pressure read + calculated mmHg
pstream	Live pressure stream with bar graph (any key stops)
battery	Battery ADC raw + voltage + percent
pins	Print all output pin states
buttons	Print all button pin states (press button while typing)

Recommended Debug Workflow

```
selftest          <- Run first. Checks every component automatically.
                     Outputs PASS/FAIL per component.

led               <- Confirm GPIO3 working. Should blink 3x.
pump on           <- Hear motor? If not: check GPIO21, check 5V rail, check wiring
pump off

v1 on             <- Hear valve click? If not:
                     1. Check GPIO26 HIGH with multimeter
                     2. Check driver circuit (MOSFET/transistor)
                     3. Check 5V rail while valve on
v1 off
(repeat v2-v5, vr)

pstream           <- Watch raw values. If all same number → sensor wiring issue
```

```
pump on           <- Watch values change while pumping
pump off

battery           <- Check voltage reading. If 0: check GPIO15 divider
```

Code Snippet — selftest function

```
void selfTest() {
  // 1. LED
  for(uint8_t i=0;i<6;i++){ digitalWrite(LED_PIN,i%2==0); delay(250); }
  digitalWrite(LED_PIN,LOW);

  // 2. Battery
  readBattery();
  Serial.printf("Battery: %.2fV  %d%%\n", g_battV, g_battPct);

  // 3. Pressure sensor
  long raw = readHX710B();
  float mmhg = rawToMmHg(raw);
  Serial.printf("Pressure: %.2f mmHg (raw=%ld)\n", mmhg, raw);

  // 4. Pump motor (2s)
  digitalWrite(PUMP_PIN, HIGH); delay(2000);
  digitalWrite(PUMP_PIN, LOW);

  // 5. Each inflate valve (1s each)
  for(uint8_t i=0;i<5;i++){
    digitalWrite(VALVE_PINS[i], HIGH); delay(1000);
    digitalWrite(VALVE_PINS[i], LOW);  delay(200);
  }

  // 6. Release valve (1s)
  digitalWrite(VALVE_PINS[5], HIGH); delay(1000);
  digitalWrite(VALVE_PINS[5], LOW);
}
```

13. Pressure Calibration Code — verveflo_pressure_cal.ino

Uses Q2HX711 library. Provides live streaming, zeroing, and two-point calibration. Run this before first use to get accurate CALIB_M and CALIB_ZERO values.

Commands

Command	Action
r	Single raw reading (average of 8)
s	Stream raw values with adj and mmHg columns
z	Zero/Tare — set current reading as 0 mmHg baseline
c 50	Calibrate at known pressure (replace 50 with gauge reading)
t	Show current CALIB_M, CALIB_B, CALIB_ZERO values
p	Single mmHg reading with current calibration
ps	Live mmHg stream with bar graph
pump on/off	Control pump to build pressure
v1 on/off	Bleed pressure via cuff 1 valve
vr on/off	Bleed via release valve

Full Calibration Procedure

```
PREREQUISITE: Have a reference pressure gauge connected to the same air line.

1. Upload verveflo_pressure_cal.ino
2. Open Serial Monitor: 115200 baud, Newline line ending

3. AT ATMOSPHERE (nothing pressurized, open to air):
   Type: z
   Serial prints: "Zero set to raw=8XXXXXXX"
   NOTE THIS NUMBER → this is your CALIB_ZERO

4. BUILD PRESSURE:
   Type: pump on
   Type: ps        (watch bar graph climb)
   Type: pump off  (stop when gauge reads a known value e.g. 50 mmHg)

5. CALIBRATE:
   Type: c 50      (use whatever your reference gauge reads)
   Serial prints:
     Known    : 50.00 mmHg
     Raw      : 8XXXXXXX
     Adjusted : XXXXXXX
     CALIB_M = 0.00000625f
     CALIB_B = 0.0000f
```

```
6. VERIFY:
   Type: ps
   Watch values — should match reference gauge at various pressures

7. COPY TO MAIN FIRMWARE:
   In verveflo.ino find:
     const float CALIB_M   = -0.00000625f;  // OLD
     long        CALIB_ZERO = 0L;           // OLD

   Replace with values from step 5 and step 3:
     const float CALIB_M   =  0.00000625f;  // NEW (positive)
     const float CALIB_B   =  0.0000f;
     long        CALIB_ZERO = 8388000L;     // NEW (from step 3)
```

Important Notes

CALIB_M must be POSITIVE for this sensor — raw increases with pressure. CALIB_ZERO must be set at true atmosphere (pump off, valves open, all pressure released). Running zero command with pressure still in system gives wrong baseline.

Debug Output Format

```
// readPressure() with debug enabled outputs:
[PRESS] raw=8880956  zero=8388000  adj=492956  mmhg=3.08  result=3.08

// Explanation:
raw    = HX710B raw 24-bit value
zero   = CALIB_ZERO (atmosphere baseline)
adj    = raw - zero (pressure delta)
mmhg   = CALIB_M * adj + CALIB_B (calculated)
result = after constrain(0-200 mmHg)

// If result=0 but mmhg is negative → CALIB_M sign wrong or zero not set
// If raw=0 → sensor not responding → check wiring
// If raw=16776xxx → overflow during fast release → normal, filtered
```

14. Valve Test Code — verveflo_valve_test.ino

Uses U8g2 library. Tests all 6 valves with live OLED feedback. OLED shows filled box = valve ON, outline box = valve OFF. Updates in real time as commands are typed.

Commands

Command	Action
v1 on/off	Fill valve 1 GPIO 26 — OLED C1 box fills
v2 on/off	Fill valve 2 GPIO 47
v3 on/off	Fill valve 3 GPIO 39
v4 on/off	Fill valve 4 GPIO 40
v5 on/off	Fill valve 5 GPIO 41
vr on/off	Release valve GPIO 42 — OLED REL box fills
all on	All 6 valves ON simultaneously
all off	All 6 valves OFF
sweep	Auto open each valve 2s in sequence with OLED animation
status	Print all valve states + GPIO readings

OLED Display Layout

```
Row 1: VALVE TEST header
Row 2-3: V1[C1/G26]  V2[C2/G47]  V3[C3/G39]   <- filled = ON
Row 4-5: V4[C4/G40]  V5[C5/G41]  VR[REL/G42]  <- outline = OFF
Row 6: "N/6 ON -- serial cmd"

Box filled = valve ON (solenoid energized)
Box outline = valve OFF (solenoid de-energized)
```

Use This When

Symptom	Test to run
Valve not clicking	sweep — confirms which valves respond
Cuff not inflating	v1 on → check GPIO HIGH + listen for click
All valves dead	all on — if nothing → check 5V rail
Specific valve not working	vN on individually → check driver circuit for that valve
Release not deflating	vr on → listen for click on release valve GPIO42
Valve stuck open	alloff → confirms firmware can control all pins

15. Troubleshooting

Serial Monitor Shows Nothing

Check	Fix
USB CDC On Boot setting	Tools → USB CDC On Boot → Enabled → re-upload
Baud rate	Set to 115200 in Serial Monitor
Wrong COM port	Tools → Port → select JTAG/CDC port
ESP32-S3 needs time	Add delay(2000) at start of setup()

Pressure Always Shows 0

Symptom	Cause	Fix
raw=0 always	Sensor not connected	Check DATA=GPIO34 CLK=GPIO33 VCC GND
mmhg negative, result=0	CALIB_M is negative, raw going up	Change CALIB_M to positive
CALIB_ZERO wrong	Zero run under pressure	Release all pressure, type zero again
raw=16777216 always	Overflow state	Power cycle sensor, check wiring
raw changes but mmhg flat	CALIB_ZERO too high	Re-run zero at true atmosphere

Buck Converter Overheating

Cause	Fix
Motor startup spike	Firmware motor soft start (updateMotorRamp) — verify it is in loop()
Cap missing	Add 100uF-2200uF across 5V output terminals
Wrong output voltage	Measure output — should be 5.0-5.2V, not higher
Too much load	Separate motor onto its own power path (MOSFET from battery)

BMS Shuts Off Under Load

Cause	Fix
Voltage sag from motor spike	Add 1000uF+ cap directly on battery output terminals
BMS cutoff too high	Check BMS cutoff voltage — should be 5.4-5.6V for 2S
Single cell battery	Ensure 2S (two cells in series) for 8.4V — single cell too low
PWM not working	Type motor 50 in serial — confirm slow ramp, not instant full speed

WiFi Not Connecting

Symptom	Fix
No VerveFlo-Setup hotspot	Type wificlear in serial → restarts in AP mode
Captive portal not opening	Manually open 192.168.4.1 in browser

Saved WiFi fails to connect	Type wificlear → redo setup with correct password
IP not showing on OLED	Type ip in serial monitor
UI shows OFFLINE	Check IP in connect field — must match device IP exactly

Valves Not Responding

Symptom	Fix
No click sound	Check driver circuit (MOSFET gate voltage), check 5V rail
Some valves work	Check specific GPIO pin HIGH with multimeter, check driver for that channel
Valve stuck open	Type vN off in serial, check if GPIO goes LOW
PWM ramp not working	Type v1 on for instant ON — if works, ramp issue
All valves dead	Check 5V rail voltage under load, check common GND